



B.TECH – COMPUTER SCIENCE

Stacks, Recursion & Queues

UNIT 2 – Stack Operations & Applications, Recursion, Queues, Circular Queues, Deque and Priority Queues



ROADMAP

What We Will Cover Today

01

Stacks — Concepts & Representation

Definition, operations, array & dynamic array implementation

02

Stack Applications

Polish notation, infix to postfix, postfix evaluation

03

Recursion

Factorial, GCD, Fibonacci, Tower of Hanoi

04

Queues — Concepts & Representation

Definition, array representation, queue operations

05

Circular Queue, Deque & Priority Queue

Circular queues, dynamic circular queues, deque, priority queue



01

Stacks — Concepts & Representation

Understanding LIFO structures, their operations and implementation



STACKS

What is a Stack?

- **Definition**

- A Stack is a linear data structure that follows the LIFO principle — Last In, First Out. The element inserted last is the first one to be removed.

- **Real-Life Analogy**

- A stack of plates — you always add a new plate on top, and always remove the topmost plate first.

- **Key Term**

- TOP — a pointer/index that always refers to the topmost element of the stack.

- **Where Stacks Are Used**

- Function call management, undo operations, expression evaluation, backtracking algorithms, browser history.



LIFO

- Last In → First Out
- Insertion & deletion happen only at ONE end — the TOP
- Opposite of a Queue (FIFO)



STACKS

Stack Operations

- **push(x)**
 - Inserts element x onto the top of the stack. Fails if the stack is full (Stack Overflow).
- **pop()**
 - Removes and returns the topmost element. Fails if the stack is empty (Stack Underflow).
- **peek() / top()**
 - Returns the topmost element WITHOUT removing it.
- **isEmpty()**
 - Returns true if the stack contains no elements ($TOP = -1$).
- **isFull()**
 - Returns true if the stack has reached its maximum capacity.



Complexity

- push $\rightarrow O(1)$
- pop $\rightarrow O(1)$
- peek $\rightarrow O(1)$
- All operations work at only the TOP, so they are all constant time



STACKS

Array Representation of Stacks

```
#define MAX 100
int stack[MAX];
int top = -1;

void push(int x) {
    if (top == MAX - 1) printf("Overflow");
    else stack[++top] = x;
}

int pop() {
    if (top == -1) printf("Underflow");
    else return stack[top--];
}
```



Explanation

- The stack is implemented using a fixed-size array along with a variable top that tracks the current top position.
- Initially top = -1 means the stack is empty.
- push() increments top first, then stores the value.
- pop() returns the value at top, then decrements top.
- Simple and fast, but SIZE IS FIXED at compile time.



STACKS

Stacks using Dynamic Arrays

```
int *stack = malloc(cap * sizeof(int));
int top = -1, cap = 4;

void push(int x) {
    if (top == cap - 1) {
        cap = cap * 2;
        stack = realloc(stack,
                        cap * sizeof(int));
    }
    stack[++top] = x;
}
```



Why Dynamic?

- A dynamic array based stack removes the fixed-size limitation of a normal array stack.
- When the stack becomes full, a NEW larger block is allocated using `realloc()` (commonly double the old capacity).
- Existing elements are automatically preserved by `realloc()`.
- Gives the flexibility of unlimited growth while still keeping $O(1)$ amortized push time.



02

Stack Applications

Polish notation, infix to postfix conversion & postfix evaluation



STACK APPLICATIONS

Polish Notation — Prefix, Infix & Postfix

- **Infix Notation**

- Operator is written BETWEEN the two operands — the normal way we write expressions. Example:
 $A + B$

- **Prefix (Polish) Notation**

- Operator is written BEFORE its operands. Example: $+ A B$

- **Postfix (Reverse Polish) Notation**

- Operator is written AFTER its operands. Example: $A B +$

- **Why Convert Infix to Postfix/Prefix?**

- Infix expressions require precedence rules and parentheses, making them hard for a computer to evaluate directly. Postfix expressions can be evaluated in a single left-to-right scan using a stack — no parentheses or precedence needed.

Σ

Example

- Infix: $(A + B) * C$
- Prefix: $* + A B C$
- Postfix: $A B + C *$



STACK APPLICATIONS

Infix to Postfix Conversion

Algorithm (using a Stack):

Scan infix left to right. Operand $\rightarrow$ add to output. '(' $\rightarrow$ push. ')' $\rightarrow$ pop until '('. Operator $\rightarrow$ pop operators with $\geq$ precedence, then push it. At the end, pop all remaining operators.

Expression: $A + B * C$

Symbol	Stack (top $\rightarrow$ right)	Output
A	—	A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
end	—	A B C * +

Precedence Rules

- () — highest
- ^ (exponent)
- * and /
- + and - — lowest

Final Postfix: $A B C * +$



STACK APPLICATIONS

Evaluation of Postfix Expression

Algorithm:

Scan postfix left to right. Operand → push onto stack. Operator → pop TWO operands, apply the operator, push the result back. The final value left in the stack is the answer.

Expression: 5 3 4 * +

Symbol	Action	Stack Contents
5	push 5	5
3	push 3	5, 3
4	push 4	5, 3, 4
*	pop 4, 3 → $3 \times 4 = 12$, push 12	5, 12
+	pop 12, 5 → $5 + 12 = 17$, push 17	17



Result

Final value left in the stack = 17

This is the result of evaluating $5 + (3 \times 4)$.



03

Recursion

Factorial, GCD, Fibonacci Sequence & Tower of Hanoi



RECURSION

What is Recursion?

- **Definition**

- Recursion is a technique in which a function calls ITSELF, directly or indirectly, to solve a smaller instance of the same problem.

- **Two Essential Parts**

- Base Case — the condition that STOPS the recursion (prevents infinite calls)
- Recursive Case — the part where the function calls itself with a smaller/simpler input

- **How It Works**

- Each recursive call is placed on the CALL STACK. When the base case is reached, the calls start returning and 'unwinding' back in reverse order.



Remember

- No base case → infinite recursion → Stack Overflow
- Recursion is closely tied to the Stack data structure



RECURSION

Factorial using Recursion

```
int factorial(int n) {  
    if (n == 0)        // base case  
        return 1;  
    return n *  
        factorial(n - 1);  
}  
  
// factorial(4)  
// = 4 * 3 * 2 * 1 * 1 = 24
```



Explanation

- Base Case: $\text{factorial}(0) = 1$
- Recursive Case: $\text{factorial}(n) = n \times \text{factorial}(n - 1)$
- Each call waits for the smaller call to return before multiplying.
- $\text{factorial}(4) \rightarrow 4 * \text{factorial}(3) \rightarrow 4 * 3 * \text{factorial}(2) \rightarrow \dots \rightarrow 4 * 3 * 2 * 1 * 1 = 24$



RECURSION

GCD using Recursion (Euclidean Algorithm)

```
int gcd(int a, int b) {  
    if (b == 0)        // base case  
        return a;  
    return gcd(b, a % b);  
}  
  
// gcd(20, 8)  
// gcd(8, 4) -> gcd(4, 0) -> 4
```



Explanation

- Based on the Euclidean Algorithm: $\text{GCD}(a, b) = \text{GCD}(b, a \bmod b)$
- Base Case: when b becomes 0, 'a' is the GCD.
- Each step reduces the numbers quickly, making it very efficient — $O(\log(\min(a, b)))$.
- $\text{gcd}(20, 8) \rightarrow \text{gcd}(8, 4) \rightarrow \text{gcd}(4, 0) \rightarrow$ returns 4



RECURSION

Fibonacci Sequence using Recursion

```
int fib(int n) {  
    if (n == 0) return 0; // base  
    if (n == 1) return 1; // base  
    return fib(n-1) +  
           fib(n-2);  
}  
  
// Sequence:  
// 0,1,1,2,3,5,8,13,21...
```



Explanation

- Two base cases: $\text{fib}(0) = 0$ and $\text{fib}(1) = 1$.
- Recursive Case: $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$
- Each call branches into TWO further calls — forms a recursion TREE, not a single chain.
- Simple but inefficient: Time Complexity = $O(2^n)$ since many sub-problems are recomputed repeatedly.



RECURSION

Tower of Hanoi

```
void hanoi(int n, char from,
           char to, char aux) {
    if (n == 0) return; // base
    hanoi(n-1, from, aux, to);
    printf("Move disk %d",n);
    printf(" %c to %c\n",from,to);
    hanoi(n-1, aux, to, from);
}
```



Explanation

- Rules: Move all n disks from source to destination using an auxiliary rod. Only one disk moved at a time; a larger disk can never sit on a smaller one.
- Recursive idea: move $(n-1)$ disks to auxiliary, move the largest disk to destination, then move $(n-1)$ disks from auxiliary to destination.
- Recurrence Relation: $T(n) = 2T(n-1) + 1$
- Minimum moves required = $2^n - 1$ (Time Complexity: $O(2^n)$)



04

Queues — Concepts & Representation

Understanding FIFO structures and their array implementation



QUEUES

What is a Queue?

- **Definition**

- A Queue is a linear data structure that follows the FIFO principle — First In, First Out. The element inserted first is the first one to be removed.

- **Real-Life Analogy**

- A queue of people waiting in line — the person who joins first is served first.

- **Key Terms**

- FRONT — points to the first element (to be removed next)
- REAR — points to the last element (most recently added)

- **Where Queues Are Used**

- CPU scheduling, printer job scheduling, handling requests in web servers, breadth-first search (BFS) in graphs.



FIFO

- First In → First Out
- Insertion happens at REAR, deletion happens at FRONT
- Opposite of a Stack (LIFO)



QUEUES

Queue Operations

- **enqueue(x)**
 - Inserts element x at the REAR of the queue. Fails if the queue is full (Overflow).
- **dequeue()**
 - Removes and returns the element at the FRONT of the queue. Fails if the queue is empty (Underflow).
- **peek() / front()**
 - Returns the front element WITHOUT removing it.
- **isEmpty()**
 - Returns true when front and rear indicate no elements are present.
- **isFull()**
 - Returns true when the queue has reached its maximum capacity.

O

Complexity

- enqueue $\rightarrow O(1)$
- dequeue $\rightarrow O(1)$
- peek $\rightarrow O(1)$
- Two ends are used, unlike a stack which uses only one



QUEUES

Array Representation of Queues

```
#define MAX 100
int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX-1) printf("Full");
    else {
        if (front == -1) front = 0;
        queue[++rear] = x;
    }
}
```



Explanation

- A simple queue is implemented using an array with two pointers: front and rear.
- Initially front = rear = -1 means the queue is empty.
- enqueue() increments rear and inserts at that position.
- dequeue() reads queue[front] and increments front.
- Drawback: once rear reaches MAX-1, no more insertion is possible even if front has empty slots — solved by Circular Queues.



05

Circular Queue, Deque & Priority Queue

Advanced queue variations and their real-world applications



ADVANCED QUEUES

Circular Queues — Why & What

- **The Problem with Simple Queues**

- In a simple array queue, once rear reaches the last index, insertion stops — even if elements have been dequeued from the front, leaving empty space unused.

- **The Solution: Circular Queue**

- The array is treated as CIRCULAR — the last position is connected back to the first position, so free space at the front can be reused.

- **Key Formulas**

- $\text{front} = (\text{front} + 1) \% \text{SIZE}$
- $\text{rear} = (\text{rear} + 1) \% \text{SIZE}$

- **Full / Empty Check**

- Queue is full when $(\text{rear} + 1) \% \text{SIZE} == \text{front}$. Queue is empty when $\text{front} == -1$.



Benefit

- Efficient memory reuse
- No wasted array slots
- Used in ring buffers, CPU/round-robin scheduling



ADVANCED QUEUES

Circular Queue — Implementation

```
int cq[SIZE], front=-1, rear=-1;

void enqueue(int x) {
    if ((rear+1)%SIZE == front)
        printf("Full");
    else {
        if (front == -1) front = 0;
        rear = (rear + 1) % SIZE;
        cq[rear] = x;
    }
}
```



Explanation

- The modulo (%) operator wraps the index back to 0 once it exceeds SIZE-1.
- This lets the queue reuse array slots freed by earlier dequeue() calls.
- dequeue() similarly updates: $\text{front} = (\text{front} + 1) \% \text{SIZE}$.
- Circular queues are the standard, efficient way to implement queues using arrays.



ADVANCED QUEUES

Circular Queues using Dynamic Arrays

- **Concept**

- Combines the circular queue technique with dynamic memory allocation — the array grows at runtime instead of being fixed in size.

- **How Resizing Works**

- When the circular queue becomes full, a new larger array (commonly double the size) is allocated using `realloc()`, and existing elements are copied over in correct FRONT-to-REAR order.

- **Why Reorder While Copying?**

- Because the old elements may be 'wrapped around' the array, a straight `memcpy` would scramble the order — elements must be copied starting from front for exactly 'count' elements.

- **Advantage**

- Combines the best of both worlds: efficient circular reuse of space AND unlimited growth when required.



ADVANCED QUEUES

Deque (Double Ended Queue)

- **Definition**

- A Deque (Double Ended Queue) is a linear data structure that allows insertion AND deletion from BOTH ends — front and rear.

- **Types of Deque**

- Input Restricted Deque — insertion allowed only at ONE end, deletion allowed at BOTH ends
- Output Restricted Deque — deletion allowed only at ONE end, insertion allowed at BOTH ends

- **Operations**

- insertFront(), insertRear(), deleteFront(), deleteRear(), getFront(), getRear()



Use Cases

- Undo/redo operations
- Sliding window problems (e.g. max in sub-array)
- Can behave as BOTH a stack and a queue



ADVANCED QUEUES

Priority Queue and its Applications

- **Definition**

- A Priority Queue is a special type of queue where each element has a PRIORITY, and elements are removed in order of priority — NOT strictly by insertion order.

- **Types**

- Ascending Priority Queue — element with the SMALLEST value is removed first
- Descending Priority Queue — element with the LARGEST value is removed first

- **Common Implementation**

- Usually implemented using a Heap data structure for efficient $O(\log n)$ insertion and removal of the highest priority element.

- **Applications**

- CPU task scheduling, Dijkstra's shortest path algorithm, Huffman coding, bandwidth management, hospital emergency queues.



Real World

- Hospital ER — critical patients treated first, not by arrival order
- OS process scheduling by task priority



Key Takeaways

- ✓ Stack = LIFO; push/pop/peek all work at ONE end (TOP) in $O(1)$ time
- ✓ Array-based stacks are fixed-size; dynamic-array stacks grow using `realloc()`
- ✓ Postfix expressions avoid precedence/parentheses — evaluated in one scan using a stack
- ✓ Recursion needs a Base Case + Recursive Case; it uses the call stack internally
- ✓ Classic recursion examples: Factorial, GCD (Euclidean), Fibonacci, Tower of Hanoi ($2^n - 1$ moves)
- ✓ Queue = FIFO; enqueue at REAR, dequeue at FRONT in $O(1)$ time
- ✓ Circular Queues reuse freed array space using the % (modulo) operator
- ✓ Deque allows both-end operations; Priority Queue serves elements by priority, not order



Thank You

Questions & Discussion